

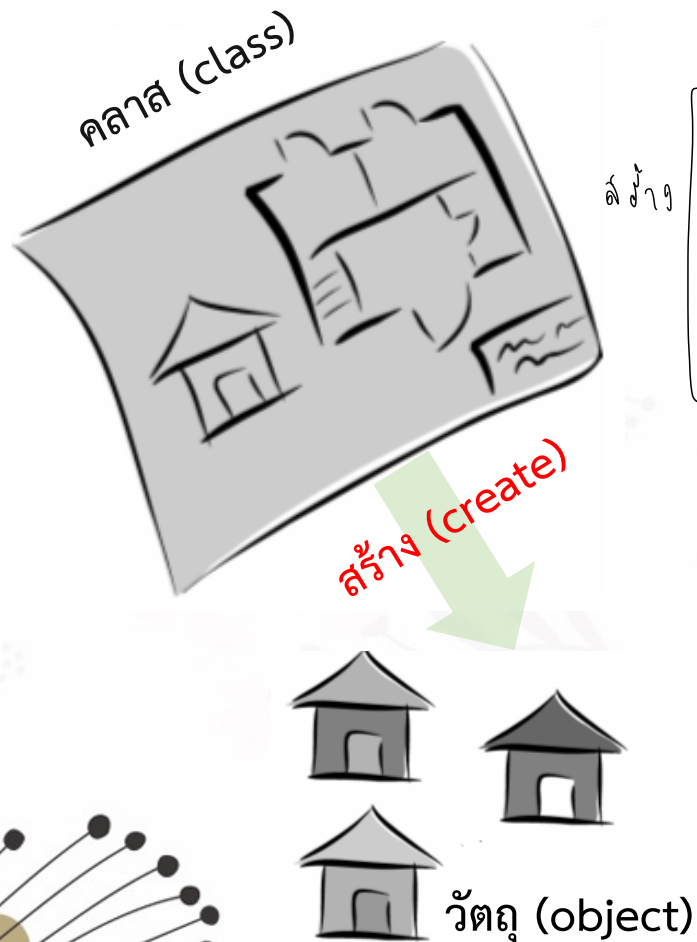


บทที่ 4

การเขียนโปรแกรมเชิงวัตถุเบื้องต้น

บรรยายโดย ผศ.ดร.ธราวิเศษฐ์ ธิติจรรณโรจน์

แบบแปลนและวัตถุที่สร้างจากแบบ

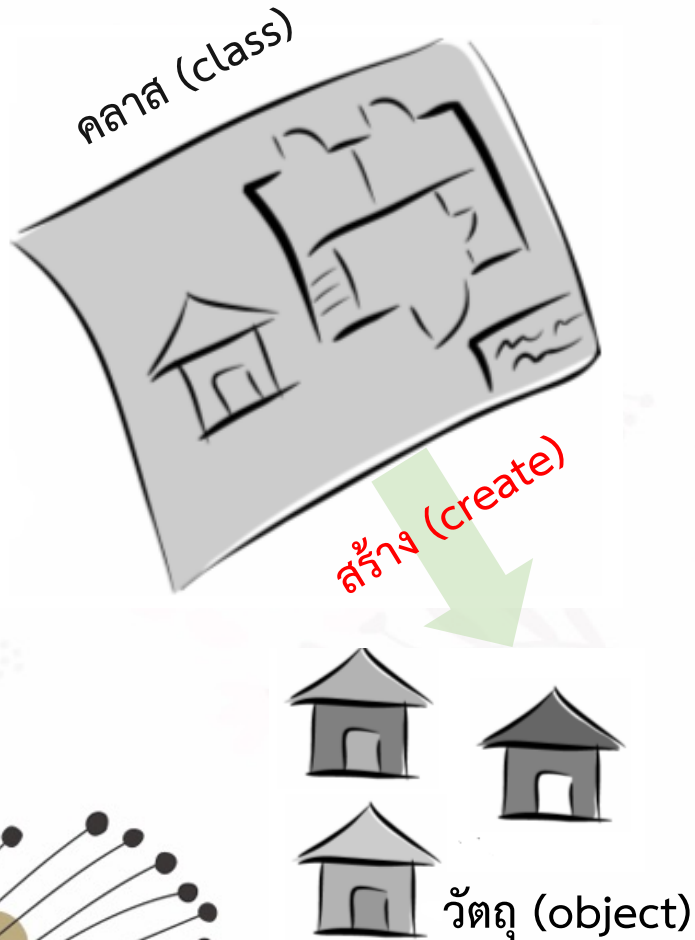


คลาส (Class) คือ แบบแผ่น หรือ แม่แบบ (Template/Blueprint) ที่ใช้สำหรับ การสร้างวัตถุ (Object) เพื่อกำหนดแอตทริบิวต์ (Attribute) และเมธอด (Method) ของวัตถุ ซึ่งเป็นตัวอธิบายรายละเอียด และหน้าที่ต่าง ๆ

วัตถุ (Object) คือ สิ่งที่มี (1) สถานะ (states) หรือแอตทริบิวต์ (Attribute) และ (2) แสดงพฤติกรรม (behaviors) หรือเมธอด (Method)

นอกจากนี้ วัตถุที่สร้างมาจากคลาส บางครั้งเรียกว่าเป็น instance ของคลาส ซึ่งคลาสหนึ่งคลาสสามารถสร้างอ็อบเจกต์ได้หลายอ็อบเจกต์ อาทิเช่น คลาสชื่อ House อาจสร้างอ็อบเจกต์ชื่อ h1, h2 หรือ h3 ซึ่งเป็นอ็อบเจกต์ชนิด House ถ้าคลาสเปรียบเสมือนชนิดสินค้าแล้ววัตถุก็เปรียบเสมือนสินค้าในแต่ละชนิด

แบบแปลนและวัตถุที่สร้างจากแบบ



ประโยชน์ของการสร้างคลาส

- จัดการหรืออ้างอิงการใช้งานในภายหลัง
- ติดป้ายบ่งบอกว่าสิ่งที่เก็บคืออะไร
- เป็นการให้ความหมายของสิ่งที่จัดเก็บ

รวม code ที่เกี่ยวข้องกันไว้ ในไฟล์เดียวกัน

การสร้างคลาสในภาษาจาวา

โดยทั่วไป จะประกอบด้วย 4 ส่วน ได้แก่

encapsulation
คีย์เวิร์ด (keyword) ที่ใช้กำหนด
ระดับการเข้าถึง (access modifier)

คีย์เวิร์ดใช้เพื่อระบุว่าเป็นการประกาศคลาส

ชื่อ Class ใช้ด้วย Capital letter

```
modifier class <Classname> {  
    [class member] body  
}
```

บริเวณที่ใช้สร้างเมธอดหรือคุณลักษณะ

ชื่อคลาส

การสร้างคลาสในภาษาจาวา

คลาสในภาษาจาวาจะถูกเขียนอยู่ในไฟล์สกุล java และชื่อไฟล์ต้องสอดคล้องกับชื่อคลาสแบบเหมือนกัน

```
public class ComplexNumber{  
  
}
```

ไฟล์ชื่อ ComplexNumber.java

* ควรจะเป็น 1 ไฟล์ / 1 คลาส

* ชื่อไฟล์ = ชื่อคลาส (ต้องเหมือนกัน)

การประกาศและสร้างอ็อบเจกต์ในภาษาจาวา

รูปแบบเต็ม

```
class ref;           // ประกาศ
ref = new class();   // สร้าง # วิธีสร้าง
# กังวลใจ # object
```

โดยที่ `ref` แทนชื่อตัวแปรสำหรับใช้อ้างอิงอ็อบเจกต์ที่สร้างขึ้นมา หรือสามารถกล่าวได้ว่าเป็นชื่อของอ็อบเจกต์ก็ได้

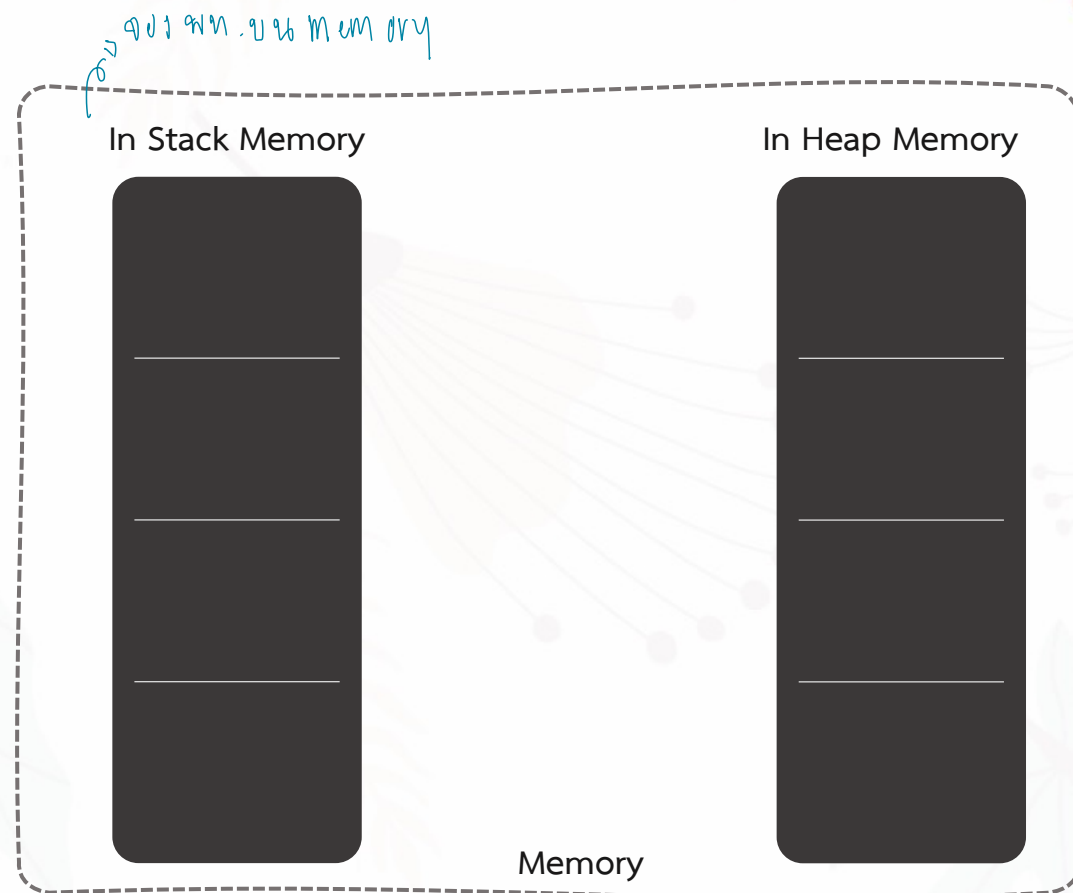
ตัวอย่าง

```
String str; # สร้าง object str จาก class String
str = new String("Alex");

ComplexNumber c; # สร้าง object c จาก class Complex Number
c = new ComplexNumber();
```


ตัวอย่าง

```
#สร้าง object 1 จาก class Sheep  
Sheep s; # ประกาศ  
  
s = new Sheep();  
  
int a;  
  
a = 10;
```



ตัวอย่าง

1

```
Sheep s;
```

```
s = new Sheep();
```

```
int a;
```

```
a = 10;
```

ตัวแปรอ้างอิง
กำหนดค่าเริ่มต้น
= null

In Stack Memory

s

null

&1102

In Heap Memory

Memory

ตัวอย่าง

```
Sheep s;
```

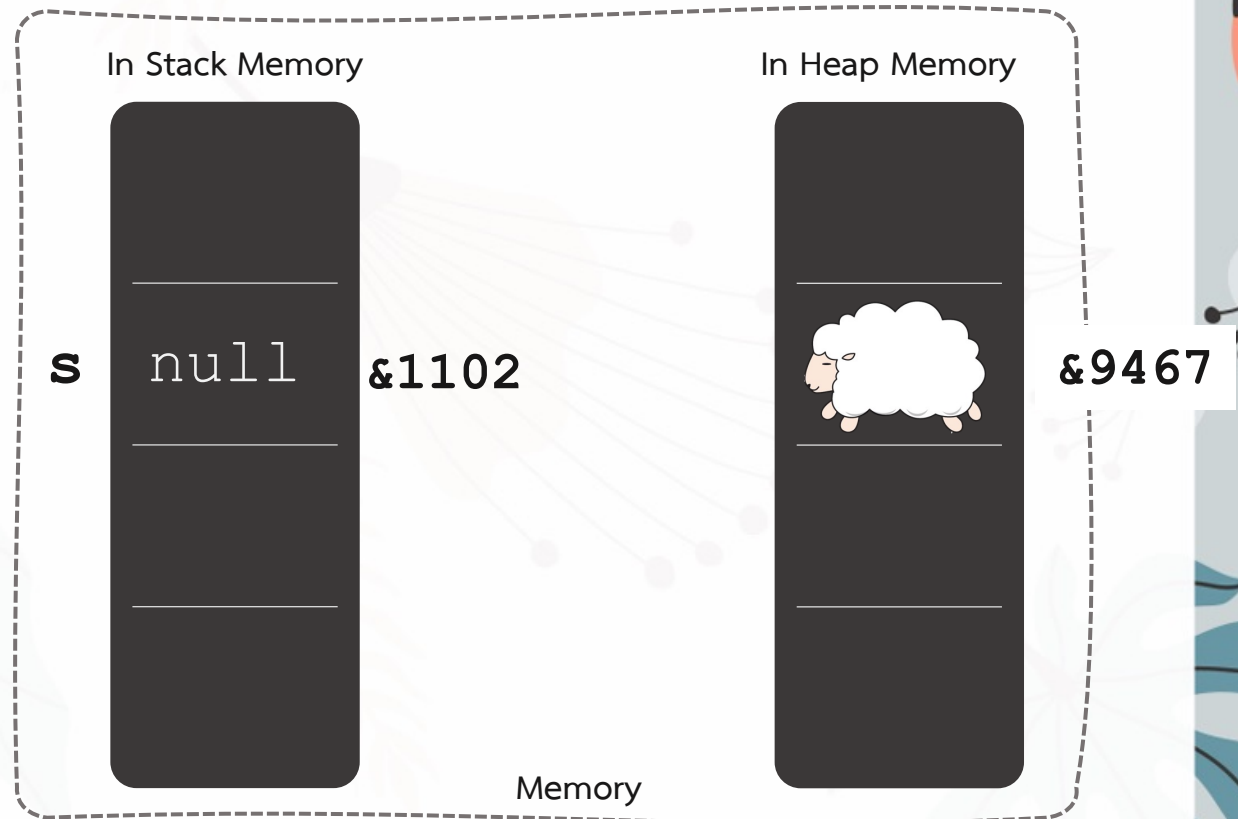
2.1

```
s = new Sheep();
```

สร้าง Object

```
int a;
```

```
a = 10;
```



ตัวอย่าง

```
Sheep s;
```

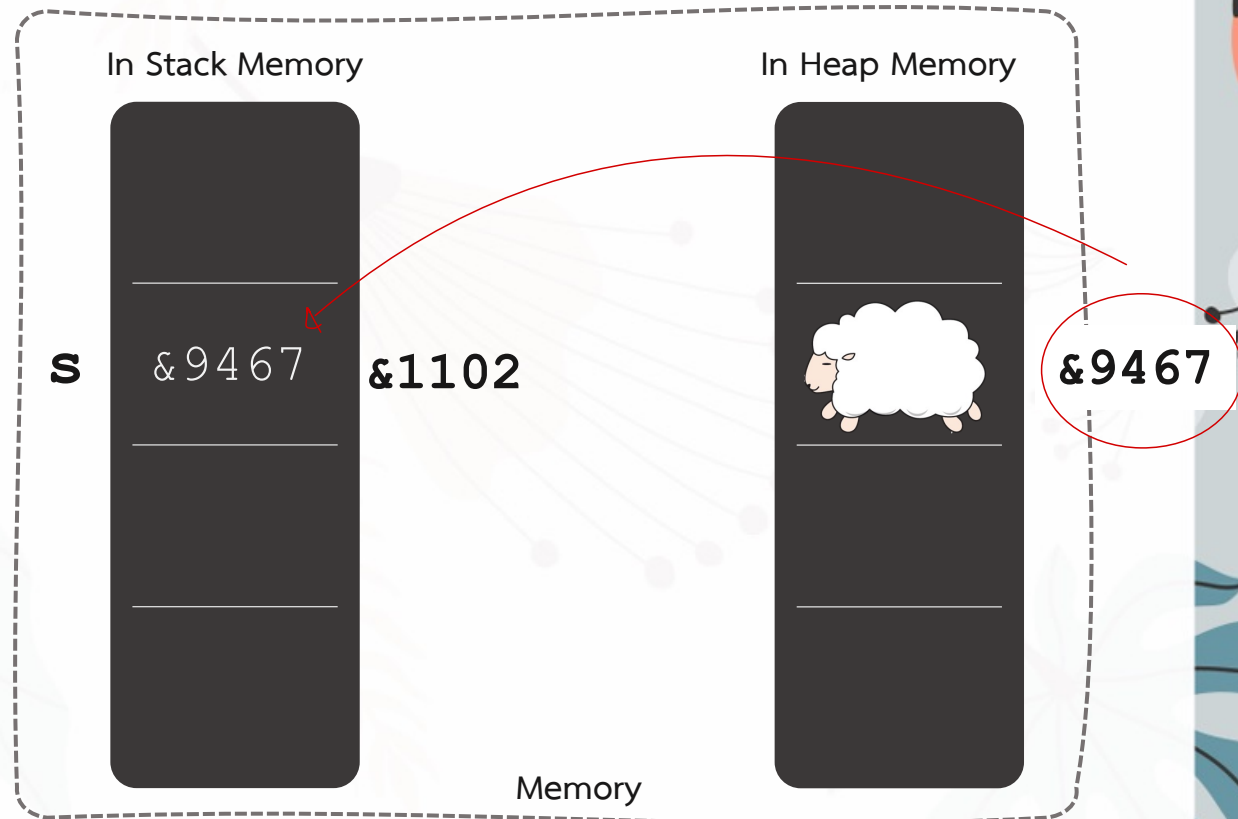
```
s = &9467;
```

2.2

```
int a;
```

```
a = 10;
```

เก็บ address เก็บไว้



ตัวอย่าง

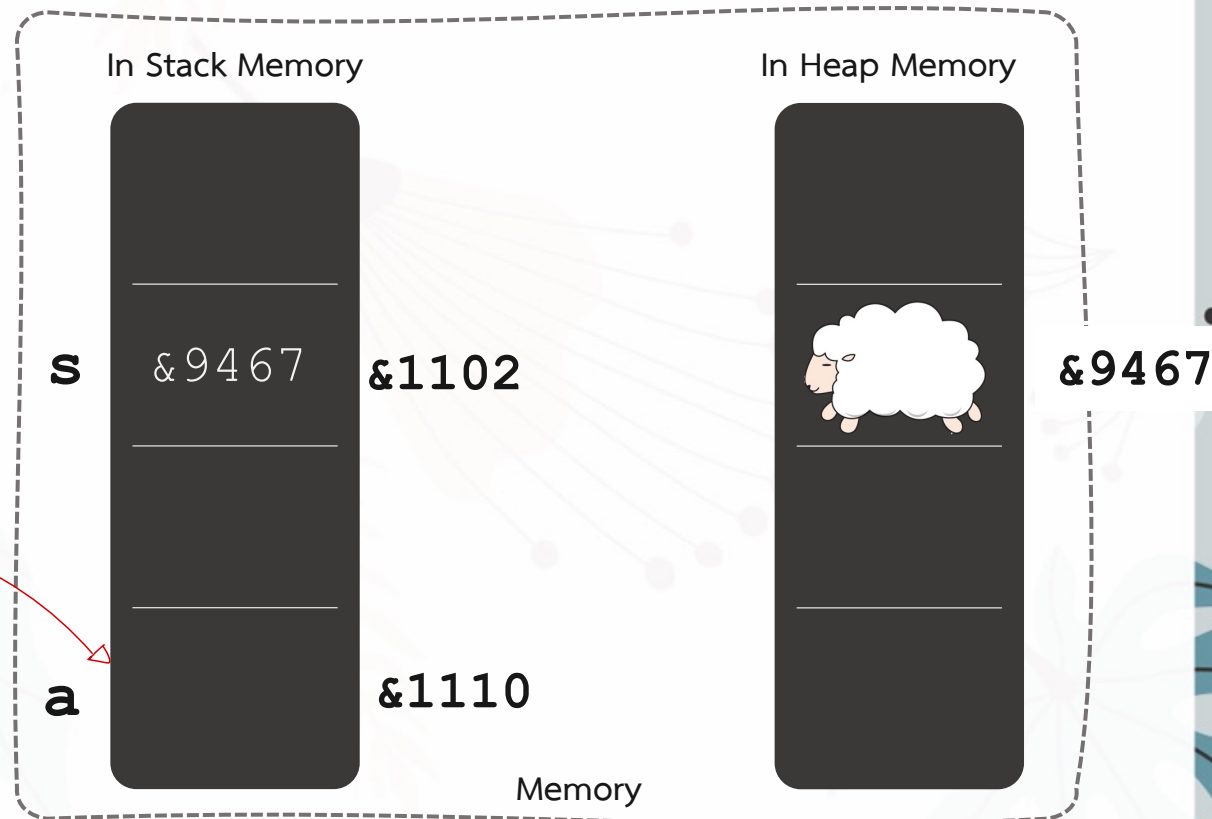
3

```
Sheep s;
```

```
s = new Sheep();
```

```
int a; จะประกาศตัวแปร  
= จองช่อง แต่ยังไม่ใส่ค่า
```

```
a = 10;
```



ตัวอย่าง

```
Sheep s;
```

```
s = new Sheep();
```

```
int a;
```

```
a = 10;
```

ประกาศไว้ในหลายๆ ครั้ง

กำหนดค่า

In Stack Memory

s

&9467

&1102

a

10

&1110

In Heap Memory



&9467

Memory

การประกาศและสร้างวัตถุในภาษาจาวา

รูปแบบย่อ

```
class ref = new class();    # ประกาศ + สร้าง object
```

ตัวอย่าง

```
    # ประกาศ + สร้าง object  
String str = new String("Alex");  
ComplexNumber c = new ComplexNumber();
```

การประกาศและสร้างวัตถุในภาษาจาวา

คลาส (Home)



ชื่อคลาส object = new ชื่อคลาส
Home myHome1 = new Home();
Home myHome2 = new Home();
Home myHome3 = new Home();
สร้าง object myHomes จากคลาส Home



myHome1



myHome2

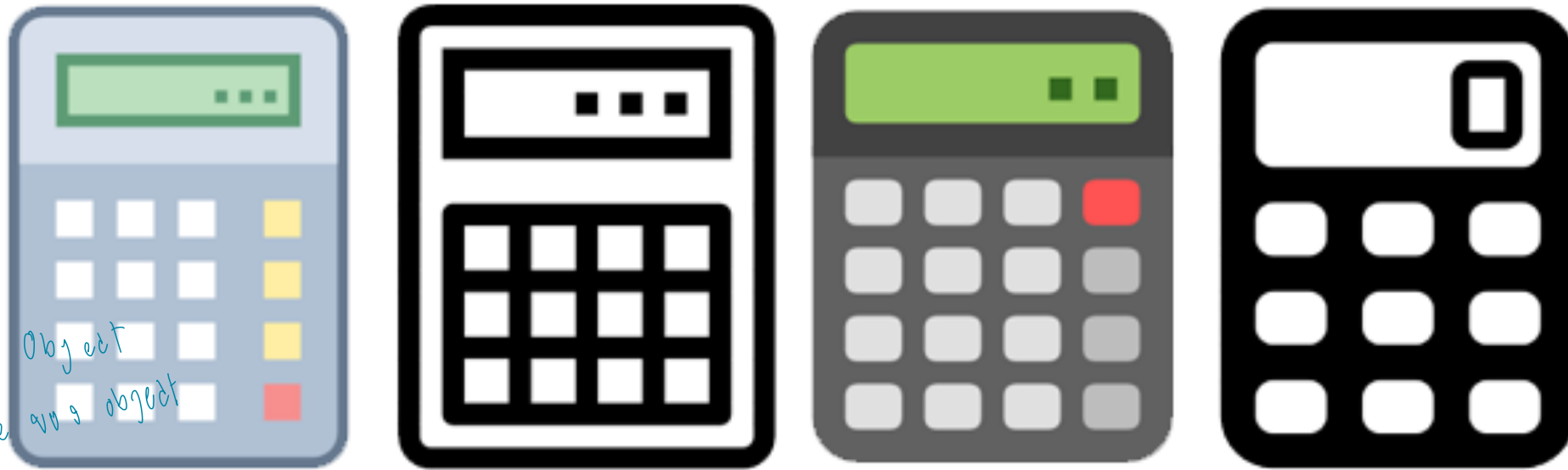


myHome3

วัตถุ (object)

ตัวแปรวัตถุ (instance variable)

ส่วนประกอบของคลาส



จัดข้อมูลของ Object
Variable ของ object
(Object มี attribute และ method)

- **แอตทริบิวต์** คือ สิ่งที่ยกบอกถึงลักษณะต่าง ๆ ของวัตถุในคลาส เช่น สี ขนาด รูปแบบฟอนต์ รุ่น เป็นต้น
- **เมธอด** คือ สิ่งที่ใช้บรรยายการทำงานของวัตถุในคลาส เช่น บวก() ลบ() คูณ() หาร() เป็นต้น

ส่วนประกอบของคลาส

Variable var object

```
public class ComplexNumber {
```

แอตทริบิวต์ (Attribute)

```
    private double realNum ;
```

```
    private double imagNum ;    # ประกาศ attribute ภายใน ชื่อจากชื่อ/ชื่อของ class
```

```
    public void myAdd (ComplexNumber c) {
```

```
        double realPart = realNum + c.realNum;
```

```
        Double imagPart = imagNum + c.imagNum;
```

```
        System.out.print(realPart + " + " + imagPart + "i");
```

```
    }
```

```
    public void myPrint() {
```

```
        System.out.print(realNum + " + " + imagNum + "i");
```

```
    }
```

```
}
```

ประกาศ method ภายใน ชื่อจากชื่อ/ชื่อของ class

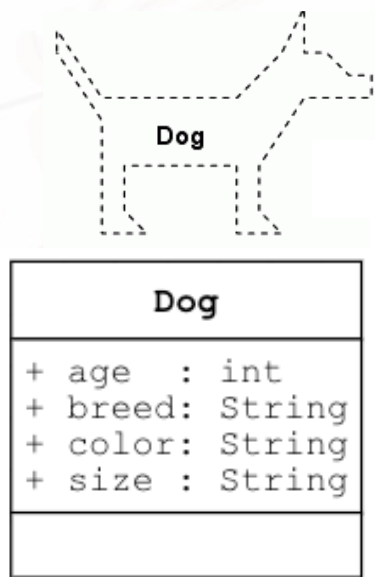
เมธอด (Method)

ตัวอย่างคลาส Math

```
public class Math{  
    public static double E = xxx;  
    public static double PI = 3.14xx;  
  
    public static double sin(double a){...}  
    public static double cos(double a){...}  
    public static double exp(double a){...}  
    public static double log(double a){...}  
    public static double sqrt(double a){...}  
    public static double ceil(double a){...}  
    public static double pow(double a, double b){...}  
    public static double abs(double a){...}  
}
```


แอททริบิวต์ (Attribute)

Attribute เน้นที่ชื่อ Attribute Value ไม่เท่ากัน



Breed = "rottweiler"
Size = "Large"
Age = 5
Color = Black



Breed = "shiba "
Size = "Large"
Age = 3
Color = Brown



Breed = "beagle"
Size = "small"
Age = 1
Color = Brown



แอททริบิวต์ คือ สิ่งที่ยบ่งบอกถึงลักษณะต่าง ๆ ของอ็อบเจกต์ของคลาส เช่น สี ขนาด รูปแบบฟอนต์ รุ่น เป็นต้น สำหรับการพัฒนาโปรแกรมเชิงวัตถุ แอททริบิวต์ คือ ข้อมูลที่เก็บอยู่ในอ็อบเจกต์ หรือข้อมูลที่ต้องการเก็บ ซึ่งสามารถแบ่งเป็นตัวแปรและ ค่าคงที่

การประกาศแอททริบิวต์ของอ็อบเจกต์

ภาษาจาวาจะสามารถประกาศแอททริบิวต์ได้ดังต่อไปนี้

```
Modifier Optional DataType AttributeName [= value];
```

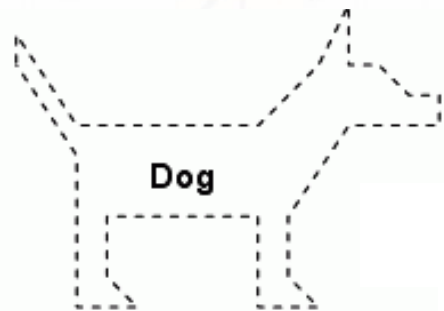
โดยที่ มีองค์ประกอบดังนี้

- Modifier คือ คีย์เวิร์ดที่ใช้กำหนดสิทธิการเข้าถึงแอททริบิวต์
- DataType คือ ชนิดข้อมูลซึ่งอาจเป็นชนิดข้อมูลพื้นฐาน หรือ ชนิดคลาส (ชนิดอ้างอิง)
- AttributeName คือ ชื่อของคุณลักษณะ

ซึ่งนักศึกษาต้องเขียนประกาศแอททริบิวต์ภายใต้ปีกกาเปิดและปิดของคลาสเท่านั้น ไม่สามารถเขียนภายในเมธอดได้

การประกาศแอตทริบิวต์ของอ็อบเจกต์

ตัวอย่างเช่น การประกาศคลาส Dog จากคลาสไดอะแกรม



Dog	
+ age	: int
+ breed	: String
+ color	: String
+ size	: String

```
public class Dog{ # class Dog
    public String breed;
    public String size; # 4 attribute
    public int age;
    public String color;
}
```

ประกาศแอตทริบิวต์ของอ็อบเจกต์
ไว้ล่วงหน้า

การเรียกใช้งานแอตทริบิวต์ของอ็อบเจกต์

รูปแบบคำสั่ง

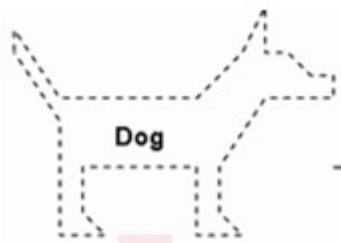
```
Class ref = new Class();  
ref.attributeName
```

ตัวอย่างการใช้งาน

```
คือ Object  
Dog d = new Dog();  
d.breed → # ใช้กับใช้  
object . Attribute
```


การเรียกใช้งานแอตทริบิวต์ของอ็อบเจกต์

ตัวอย่างเช่น การประกาศคลาส Dog จากคลาสไดอะแกรม



myDog1

มองว่า
เป็นตัวอย่าง
ปกติได้เอง

```
public class Main{  
    public static void main (String [] args){  
        Dog myDog1 = new Dog();  
        myDog1.breed = "rottweiler";  
        myDog1.size = "Large";  
        myDog1.age = 5;  
        myDog1.color = "Black";  
    }  
}
```

ตัวแปรและแอททริบิวต์

```
public class Main{  
    // แอททริบิวต์  
    public int num1; # ประกาศไว้ในไฟล์ Main = After byte  
  
    public static void main (String [] args){  
  
        // ตัวแปรในเมธอด main  
        int num2;  
    } # ประกาศไว้ในไฟล์ Method เป็นตัวแปรแบบ Local  
}
```

ค่าเริ่มต้นของตัวแปรและแอททริบิวต์

```
public class Main{  
    // แอททริบิวต์ของคลาส  
    public int num1;  
    public static void main (String [] args){  
        // ตัวแปรในเมธอด main  
        int num2; # ตัวแปรที่ไม่ได้ประกาศค่าเริ่มต้น = ไม่มีการตั้งค่า = ERROR!  
        System.out.println(" num : " + num2 );  
    }  
}
```

```
Main.java:7: error: variable num2 might not have been initialized  
    System.out.println(" num : " + num2 );  
                                ^
```

1 error

ค่าเริ่มต้นของตัวแปรและแอททริบิวต์

```
public class Main{  
    // แอททริบิวต์ของคลาส  
    public int num1; # Attribute ที่ไม่กำหนดค่าเริ่มต้น = ค่าเริ่มต้น คือ 0 / 0.00  
    public static void main (String [] args){  
        // ตัวแปรในเมธอด main  
        int num2;  
        Main m = new Main();  
        System.out.println(" num : " + m.num1 );  
    }  
}
```

num : 0

ค่าเริ่มต้นของตัวแปรและแอททริบิวต์

```
public class Main {  
    public double num;  
    public String name;  
    public char blood;  
    public boolean hasPhone;
```

```
    public static void main(String[] args) {  
        Main m = new Main();  
        System.out.println(" num : " + m.num );  
        System.out.println(" name : " + m.name );  
        System.out.println(" blood : " + m.blood );  
        System.out.println(" hasPhone : " + m.hasPhone );
```

```
        //System.out.println(" num : " + new Main().num );  
        //System.out.println(" name : " + new Main().name );  
        //System.out.println(" blood : " + new Main().blood );  
        //System.out.println(" hasPhone : " + new Main().hasPhone );  
    }  
}
```

ชื่อ & Attribute มาจาก Object และชื่อ

สร้าง object มาจาก class Main
เพิ่มให้ object เติบโต คือ m

* คำสั่งสำหรับ
Attribute ที่
โปรแกรม

num : 0.0
name : null
blood :
hasPhone : false

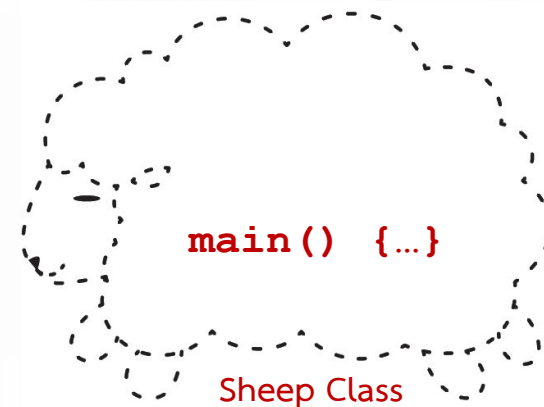
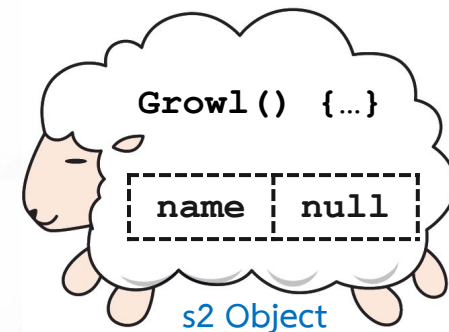
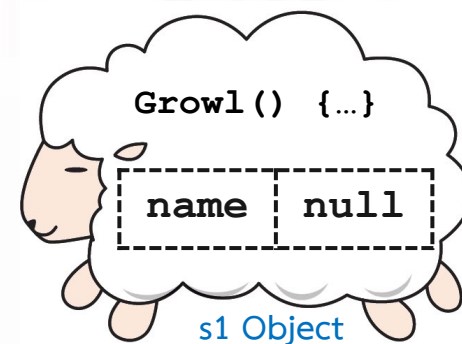
↓
ตัวแปร

การเรียกใช้งานแอททริบิวต์ของอ็อบเจกต์

การเรียกใช้แอททริบิวต์จากเมธอด ภายในคลาส

```
public class Sheep {  
    public String name; # Attribute 1 ชื่อว่า name  
    public void Growl() {  
        System.out.print("Growl Growl " + name);  
    }  
    public static void main(String[] args) { # Attribute  
        Sheep s1 = new Sheep(); # สร้าง Object 1  
        s1.Growl(); # เรียกใช้  
        Sheep s2 = new Sheep(); # สร้าง Object 2  
        s1.Growl(); # เรียกใช้  
    }  
}
```

Growl Growl null
Growl Growl null



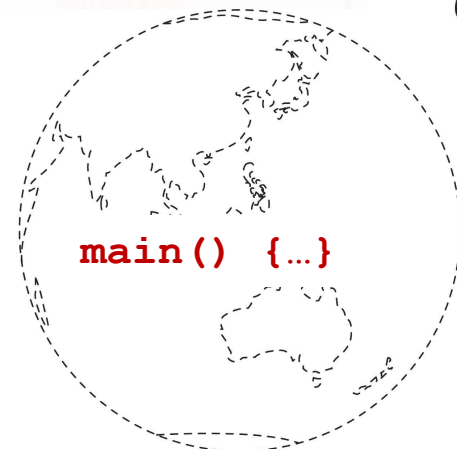
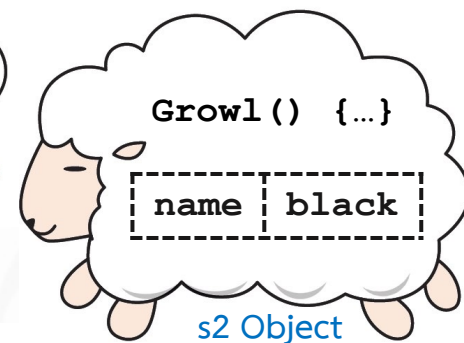
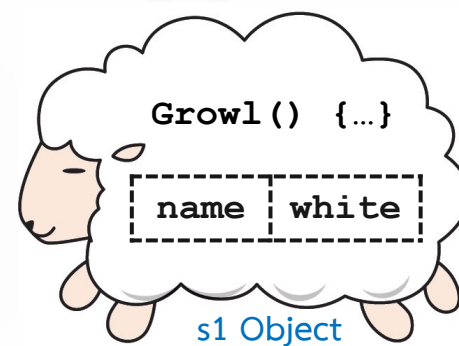
การเรียกใช้งานแอตทริบิวต์ของอ็อบเจกต์

การเรียกใช้แอตทริบิวต์จากเมธอด ภายนอกคลาส

```
public class Sheep {  
    public String name;  
    public void Growl() {  
        System.out.print("Growl Growl " + name);  
    }  
}  
  
public class Earth {  
    public static void main(String[] args) {  
        Sheep s1 = new Sheep();  
        s1.name = "white";  
        Sheep s2 = new Sheep();  
        s2.name = "black";  
    }  
}
```

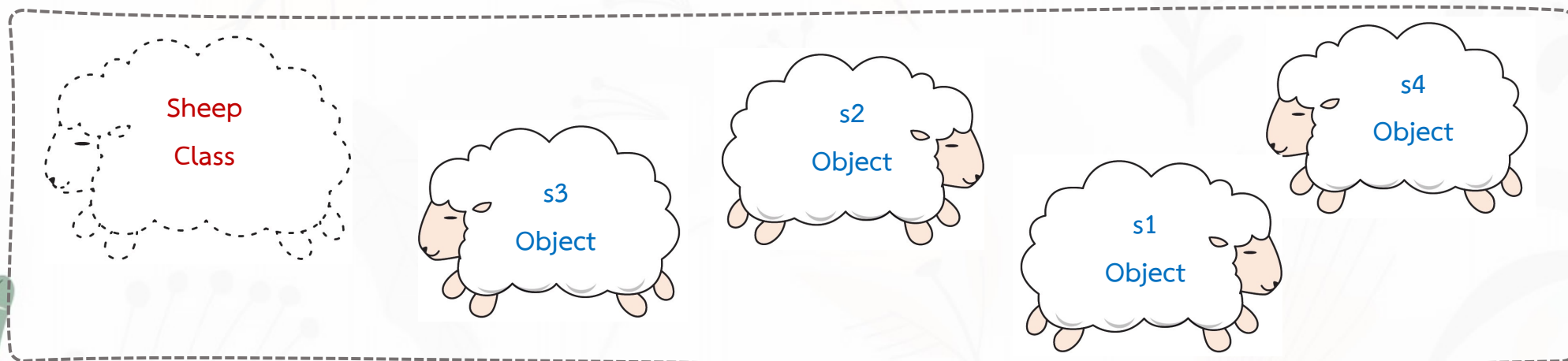
ชี้ถึง Attribute ใน object ที่ถูกเรียกใช้แล้ว

Attribute ไม่ได้อยู่ใน object. Attribute



Earth Class

การเรียกใช้งานแอตทริบิวต์ของอ็อบเจกต์

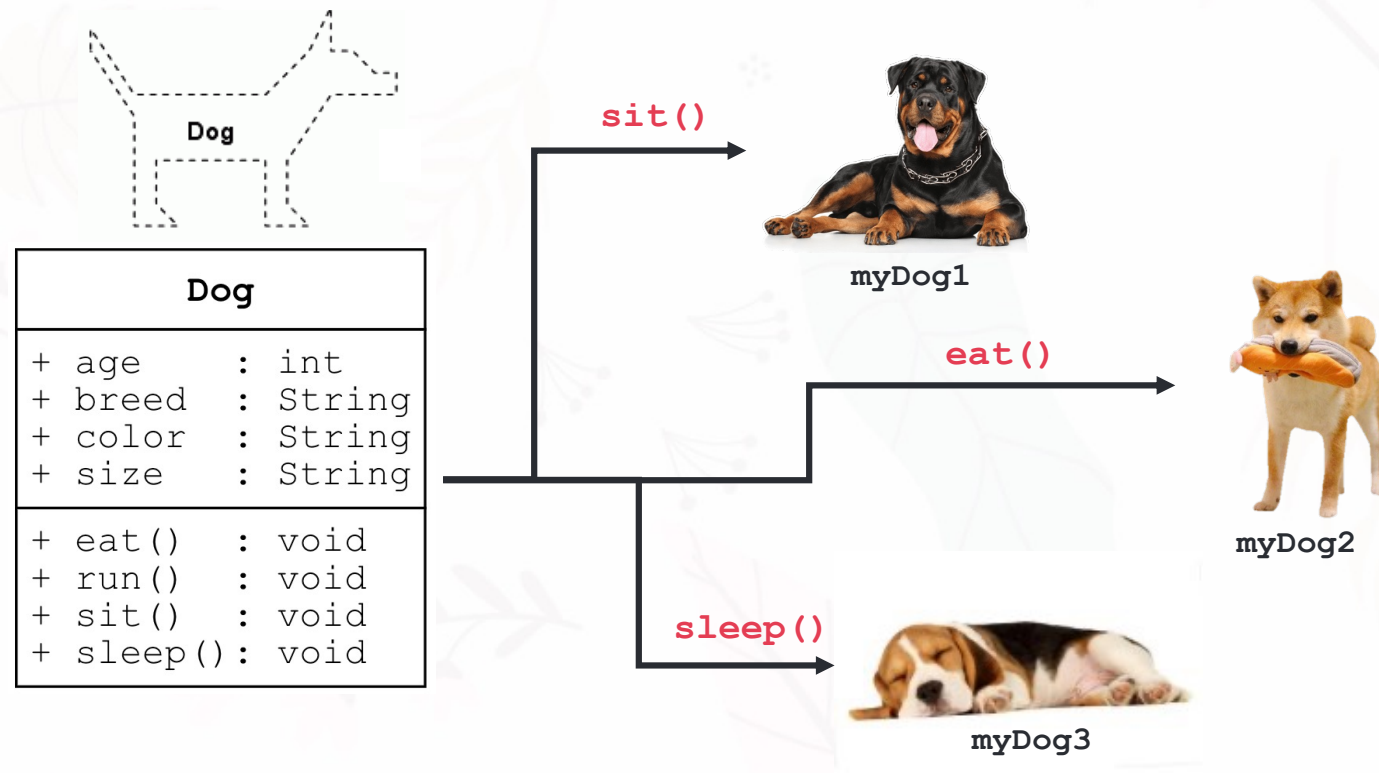


นักศึกษาต้องระบุว่าการแอตทริบิวต์ของใคร (who)

who.attributeName

- ไม่ระบุ (โดยจาวาจะพิจารณาจากอ็อบเจกต์ก่อนไปคลาส)
- ชื่อคลาส * จั๋วไข่เหิงน
- ชื่ออ็อบเจกต์
- คีย์ this หรือ คีย์ super * จั๋วไข่เหิงน

เมธอด (Method)

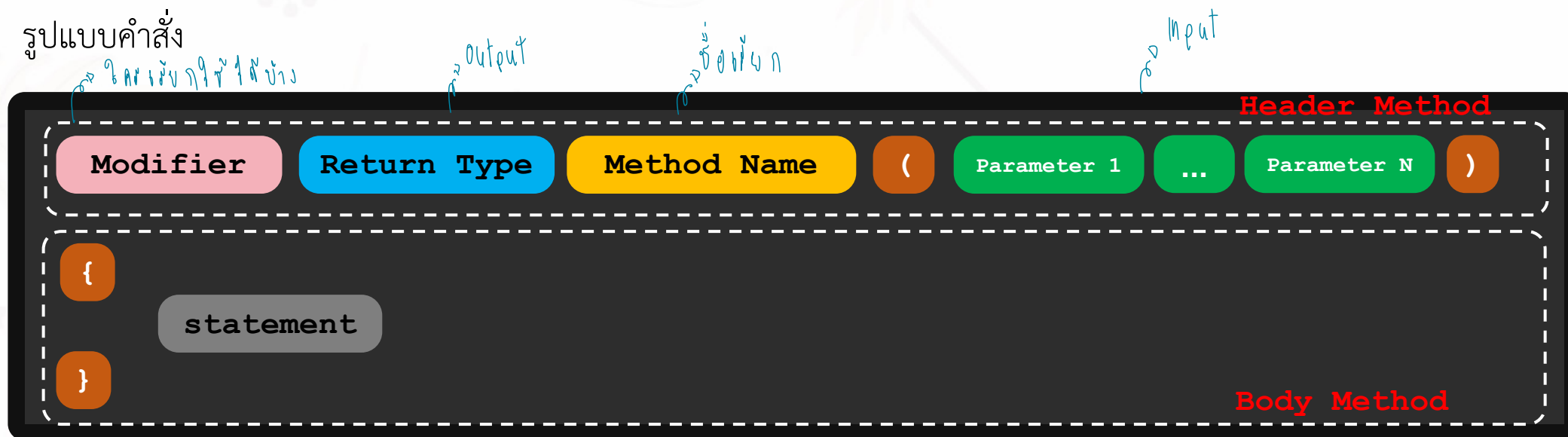


เมธอด คือ (1) **สิ่งที่อธิบายการทำงานของอ็อบเจกต์ของคลาส** เช่น eat() sleep() sit() run() เป็นต้น หรือ (2) **วิธีการหรือการกระทำที่นิยามอยู่ในคลาสหรืออ็อบเจกต์เพื่อใช้ในการจัดการกัตคุณลักษณะของอ็อบเจกต์** ซึ่งเปรียบเทียบกับ function, procedure หรือ subroutine ของโปรแกรมเชิงกระบวนการ

- เมธอดจะทำงานก็ต่อเมื่อมีการเรียกใช้งาน
- ชื่อของเมธอดควรเป็นคำกริยา

การสร้างเมธอดของอ็อบเจกต์

รูปแบบคำสั่ง



เมธอด ประกอบไปด้วย 2 ส่วนหลัก ได้แก่ Header และ Body ซึ่งในส่วน Header จะใช้เพื่อระบุ (1) การกำหนดการเข้าถึง, (2) กำหนดการคืนค่า, (3) ชื่อเมธอด และ (4) พารามิเตอร์ของเมธอด ขณะที่ส่วน Body ใช้ระบุขั้นตอนการทำงานของเมธอด

การสร้างเมธอดของอ็อบเจกต์

นักศึกษาสามารถสร้างเมธอดได้ 4 แบบ โดยแบ่งตามการรับค่าและการคืนค่า

```
public class Main{  
    public void printName(){  
        System.out.print("Taravichet");  
    }  
    public String getName(){  
        return "Taravichet";  
    }  
    public void prtName(String name){  
        System.out.print(name);  
    }  
    public int add(int a, int b){  
        return (a+b);  
    }  
}
```

// 1 ไม่รับ input
ไม่คืน output

// 2 ไม่รับ input
คืน output string

// 3 รับ input string
ไม่คืน output

// 4 รับ input int
คืน output int

* void เป็น keyword ที่บอกว่า
จะไม่มีการ return ค่ากลับ

อย่างไรก็ตาม การคืนค่า (return) คือการคืนค่าให้ผู้เรียกใช้งานนำค่าไปใช้งานต่อ ซึ่งไม่เท่ากับการใช้คำสั่ง print ()

การสร้างเมธอด

```
public class Dog{ # class Dog
    public String breed;
    public String size; # 4 attribute
    public int age;
    public String color;
    public void sit(){ # method
        System.out.print("Sit...");
    }
    public void eat(){ # method
        System.out.print("Eat...");
    }
    public void sleep(){ # method
        System.out.print("Sleep...");
    }
    public void run(){ # method
        System.out.print("run...");
    }
}
```


การเรียกใช้เมธอดของอ็อบเจกต์

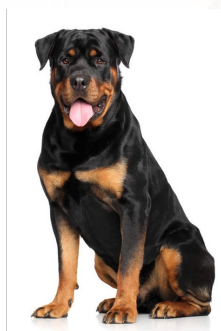
รูปแบบของคำสั่งที่ใช้เรียกใช้เมธอดเป็นดังนี้

```
Class ref= new Class();  
ref.methodName([arguments]);  
object . method
```

โดยที่ arguments อาจจะเป็นข้อมูลค่าคงที่หรือตัวแปร นอกจากนี้ ชนิดข้อมูลของ arguments ที่ใช้ในการเรียกเมธอด จะต้องสอดคล้องกันกับชนิดข้อมูลของ arguments ของเมธอด

การเรียกใช้เมธอดของอ็อบเจกต์

ตัวอย่างเช่น การเรียกใช้งานเมธอดของอ็อบเจกต์ที่สร้างมาจากคลาส Dog



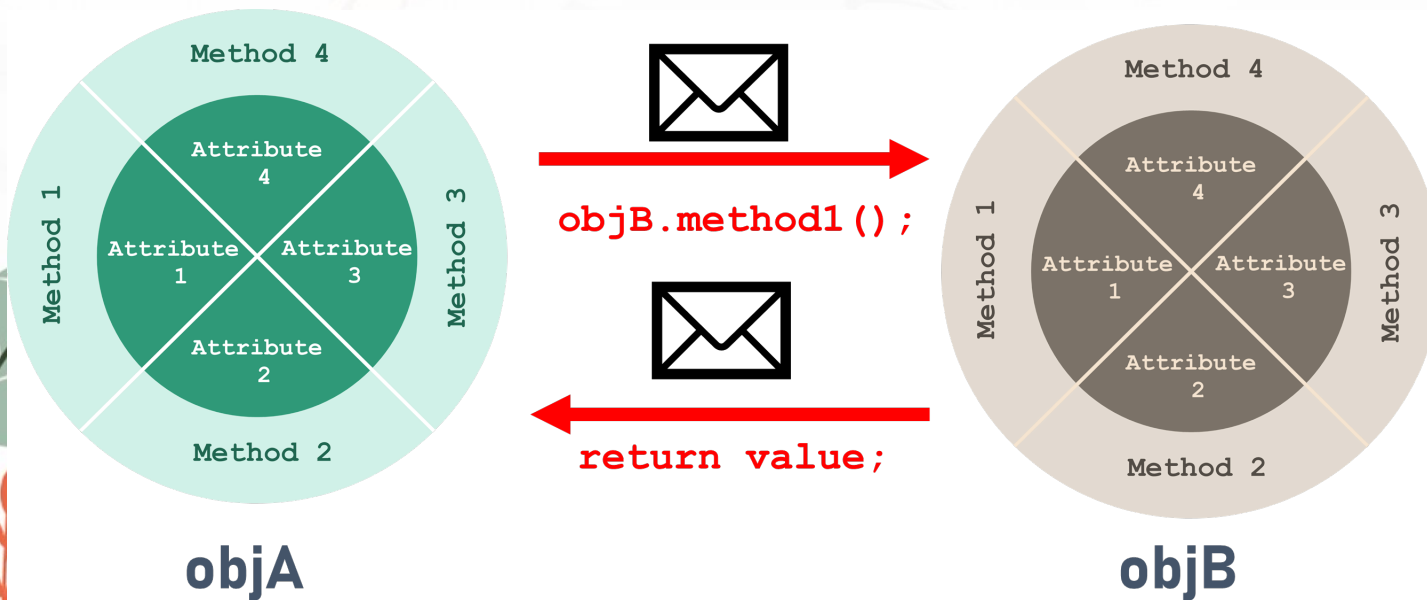
`myDog1.sit();`



`myDog1`

```
public class Main{  
    public static void main (String [] args) {  
        Dog myDog1 = new Dog();  
        myDog1.breed = "rottweiler";  
        myDog1.size = "Large";  
        myDog1.age = 5;  
        myDog1.color = "Black";  
  
        myDog1.sit();  
    }  
}
```

การเรียกใช้เมธอดของอ็อบเจกต์



การเรียกใช้เมธอดของอ็อบเจกต์ ถือว่าเป็น การสื่อสารระหว่างอ็อบเจกต์ กล่าวคือเป็น การรับส่งข้อมูลระหว่างอ็อบเจกต์ หรือ ภายในอ็อบเจกต์ ตัวอย่างเช่น ข่าวนสารจะ ส่งผ่าน method 3 จากอ็อบเจกต์ objA ที่ เป็นผู้ส่ง (sender) เพื่อเรียกการทำงานของ เมธอดที่ชื่อ method1 จากอ็อบเจกต์ objB ที่เป็นผู้รับ (receiver) ซึ่งobjB อาจส่งค่า (return value) บางค่ากลับมายัง objA

ตัวอย่างการเรียกใช้เมธอดของอ็อบเจกต์

```
public class Player{
    public int healthPoint; # 2 Attribute
    public int attack;
    public void hi () { # Method 1 → ไม่รับค่า ไม่คืนค่า
        System.out.print("Hi, you");
    }
    public void cut (Player p) { # Method 2 → รับค่า object p ไม่คืนค่า
        p.healthPoint -= attack;
    }
}

public class Main {
    public static void main (String [] args) {
        Player p1 = new Player();
        Player p2 = new Player();
        p1.hi();
        p2.hi();

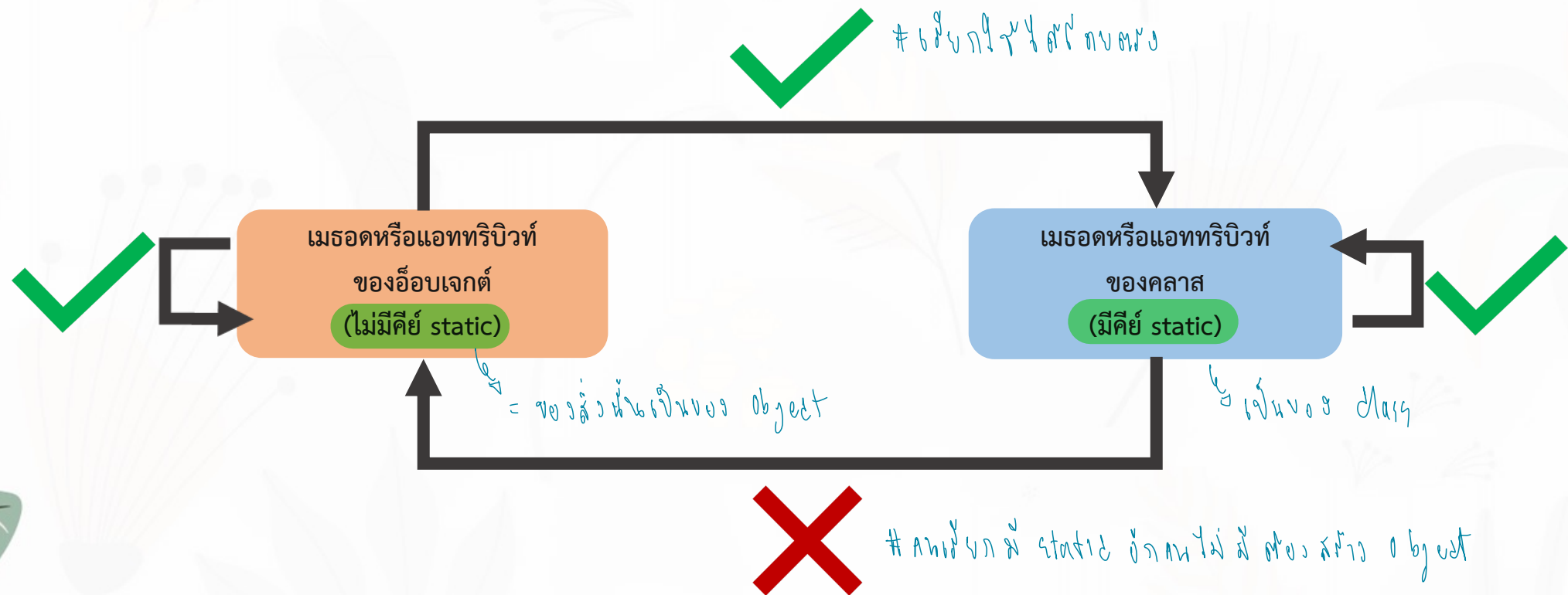
        // การสื่อสารระหว่างอ็อบเจกต์ *
        P1.cut (P2);
    }
}
```

Diagram annotations:

- Handwritten p_2 with an arrow pointing to the `p` parameter in the `cut` method.
- Handwritten p_1 with an arrow pointing to the `attack` attribute access in the `cut` method.
- Handwritten p_2 with an arrow pointing to the `p` parameter in the `cut` method.
- Handwritten p_1 with an arrow pointing to the `attack` attribute access in the `cut` method.
- Handwritten "Object, method" with an arrow pointing to the `cut` method call in the `Main` class.

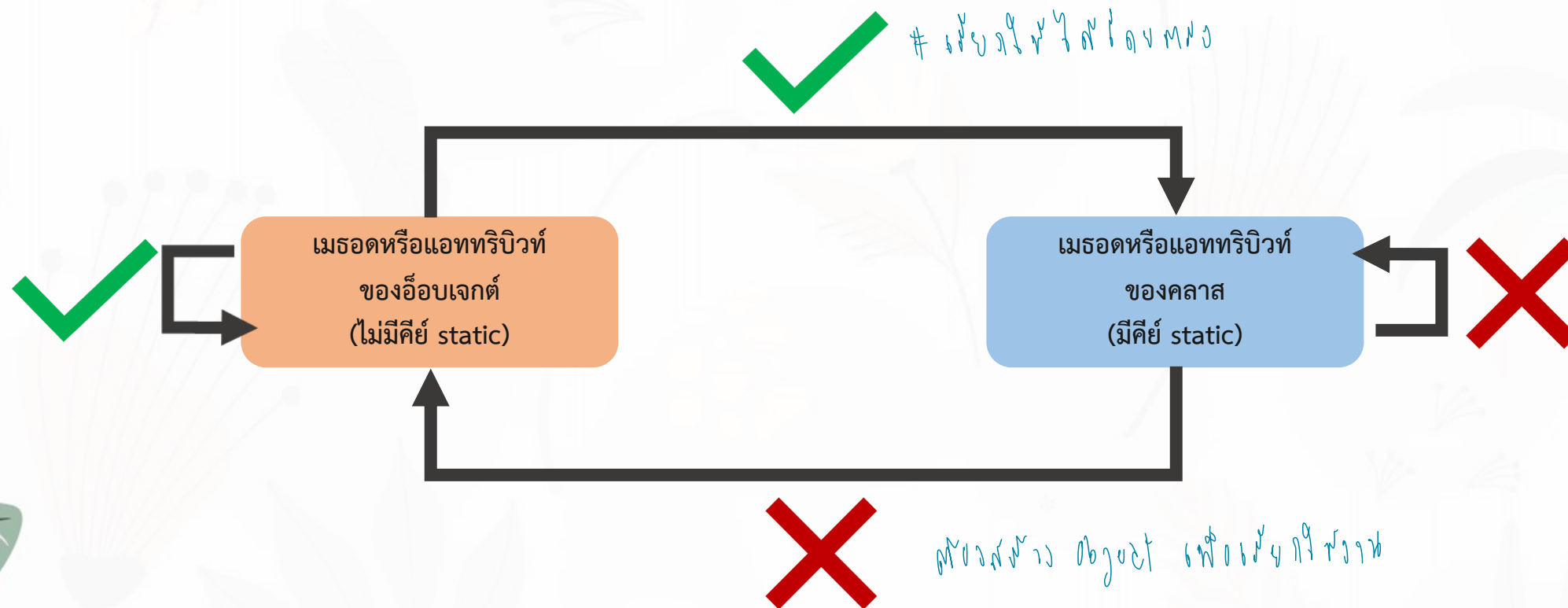
การเรียกใช้เมธอดหรือแอททริบิวต์

การเรียกใช้เมธอดภายในคลาสเดียวกัน



การเรียกใช้เมธอดหรือแอททริบิวต์

การเรียกใช้เมธอดระหว่างคลาส



การเรียกใช้เมธอดในคลาสเดียวกัน

```
public class Numerical {  
    public void calculate(double score) { # ใส่ค่า ไม่คืน  
        if (score >= 80)  
            System.out.println("Grade is A");  
        else if (score >= 70)  
            System.out.println("Grade is B");  
        else if (score >= 60)  
            System.out.println("Grade is C");  
        else if (score >= 50)  
            System.out.println("Grade is D");  
        else  
            System.out.println("Grade is F");  
    }  
    public void callMethod() { # ไม่รับ ไม่คืน  
        calculate(100); // ภายในคลาสเดียวกันและเป็นเมธอดของอ็อบเจกต์ตัวเดียวกัน ดังนั้น จึงสามารถเรียกใช้งานได้เลย  
    }  
    public static void main(String args[]) {  
        Numerical num = new Numerical();  
        double score = Math.random()*100;  
        num.calculate(score); // ภายในคลาสเดียวกัน แต่เป็นเมธอดของคลาส ดังนั้น จึงสามารถเรียกใช้งานได้โดยตรง แต่ต้องเรียกใช้งานผ่านอ็อบเจกต์  
        num.callMethod();  
    }  
}
```

2 Class ไม่ static
ให้ใช้ให้ตรงไป

ใส่ค่าให้
อ็อบเจกต์ไม่คืน

สร้าง object เพื่อใช้ให้

การเรียกใช้เมธอดต่างคลาส

```
public class Numerical {
    public void calculate(double score) {
        if (score >= 80)
            System.out.println("Grade is A");
        else if (score >= 70)
            System.out.println("Grade is B");
        else if (score >= 60)
            System.out.println("Grade is C");
        else if (score >= 50)
            System.out.println("Grade is D");
        else
            System.out.println("Grade is F");
    }
}

public class Main {
    public static void main(String args[]) {
        Numerical obj = new Numerical();
        double score = Math.random()*100;
        obj.calculate(score);
    }
}
```

ตัวอย่าง

```
public class Main{
    public static void printMe(){
        System.out.println(" Me ");
    }
    public void printIT(){
        System.out.println(" IT ");
    }
    public static void main(String[] args) {
        House h = new House();
        h.printSize();           // (1)
        h.printHouseID();        // (2)
        h.printDetail1();         // (3)
        h.printDetail2();         // (4)
        h.printHi();             // (5)
        printSize();             // (6)
        printHi();               // (7)

        printMe();               // (8)
        Main m = new Main();
        m.printIT();             // (9)
    }
}
```

```
public class House{
    public static int size ; # จำนวนบ้านทั้งหมด
    public int house_id ;

    public static void printHi(){
        System.out.println(" Hi ");
        printSize();
    }
    public static void printSize(){
        System.out.println("Size "+ size);
    }
    public void printHouseID(){ # พิมพ์ ID ของบ้าน
        System.out.println("House "+ house_id);
    }
    public void printDetail1(){
        printSize();
    }
    public void printDetail2(){
        printHouseID();
    }
}
```


การส่งค่าเข้าไปในเมธอด (Argument)

พารามิเตอร์ (parameter) และ อาร์กิวเมนต์ (argument) คือ ตัวแปรที่ใช้สำหรับการรับและส่งค่าตัวแปรของระหว่างวัตถุ ผ่านทางเมธอดต่าง ๆ ซึ่ง parameter และ argument คือ สิ่งเดียวกัน แต่เราจะใช้คำว่า

- Parameter ก็ต่อเมื่อค่าตัวแปรเข้ามาในเมธอด (อยู่ในเมธอด)
- Argument จะเรียกใช้ตอนส่งค่าเข้าเมธอด (ตอนเรียกใช้งาน)

```
public class Main {  
    public void call(parameterString name) {  
        System.out.println("Hi ," + name);  
    }  
    public static void main(String args[]) {  
        Main m = new Main();  
        m.call("Bank");  
    }  
}
```

if we give parameter to the method then it is argument

user give it = Argument

กรณีที่เมธอดมี argument ที่จะรับค่าเพื่อนำไปใช้ในเมธอด อาทิเช่น คำสั่งที่เรียกใช้เมธอด call(...) จะต้องส่ง argument ที่มีชนิดข้อมูลเป็น String ไปด้วย

การส่งค่าเข้าไปในเมธอด (Argument)

อาร์กิวเมนต์สำหรับชนิดข้อมูลพื้นฐาน เราสามารถที่จะส่งค่าคงที่ข้อมูล ตัวแปร และ นิพจน์

```
Animal a1 = new Animal ();  
a1.setAge(5);           // ค่า  
  
int a = 5;  
a1.setAge(a);           // ตัวแปร  
  
a1.setAge(4+2);         // นิพจน์  
a1.setAge(4+a);         // นิพจน์
```

นอกจากนี้ อาร์กิวเมนต์สำหรับชนิดข้อมูลวัตถุ หรือ แบบอ้างอิง เราจะต้องส่งอ็อบเจกต์ที่มีชนิดข้อมูลที่สอดคล้องไปเท่านั้น ยกเว้นกรณีที่ argument นั้นมีชนิดข้อมูลเป็น String ซึ่งในกรณีนี้จะสามารถส่งข้อมูลค่าคงที่ได้

การเปลี่ยนแปลงค่าของ Argument

pass by reference

อ้างอิง

1
cup = 

fillCup()

2
cup = 

fillCup()

3
cup = 

fillCup()

pass by value

พื้นฐาน

cup =  → real

fillCup( → copy)

cup = 

fillCup()

cup = 

fillCup()

- การส่ง argument ที่มีชนิดข้อมูลเป็น **แบบพื้นฐาน** ^{เก็บ Value} หากมีการเปลี่ยนแปลงค่าของ argument ภายในเมธอดที่ถูกเรียกใช้งาน **จะไม่มีผลทำให้ค่าของ argument ที่ส่งไปเปลี่ยนค่าไปด้วย**

- การส่ง argument ที่มีชนิดข้อมูลเป็น **แบบอ้างอิง** ^{เก็บ Address} จะเป็นการส่งตำแหน่งอ้างอิงของอ็อบเจกต์ไปให้กับเมธอดที่ถูกเรียกใช้งาน ดังนั้น การเปลี่ยนแปลงค่าของคุณลักษณะของอ็อบเจกต์จะมีผล **ทำให้ค่าของคุณลักษณะของอ็อบเจกต์ที่ส่งไปเปลี่ยนไปด้วย**

* ก. ส่งค่าส่งเข้าไปใน method

ถ้าส่งแบบเก็บที่พื้นฐาน เช่น เปลี่ยนเลข หรือ ส่งค่าเลขเข้าไป

* ส่งค่าแบบเก็บ ref. เข้าไป ส่งค่าไปเก็บที่เก็บค่าของตัวแปร

การส่งข้อมูลชนิดพื้นฐานเข้าเมธอดของอ็อบเจกต์

ตัวอย่างการส่งข้อมูลชนิดพื้นฐานเข้าเมธอดของอ็อบเจกต์

```
public class Main { # class Main
    public static void main(String[] args) {
        int s = 10;
        System.out.println(s);
        DemoArg d1 = new DemoArg();
        d1.changeNumber(s);
        System.out.println(s);
    }
}
```

```
public class DemoArg { # class DemoArg
    public void changeNumber(int s){
        s = 1000;
        // System.out.println(s);
    }
}
```

Output:

10

// 1000

10

การส่งข้อมูลชนิดอ้างอิงเข้าเมธอดของอ็อบเจกต์

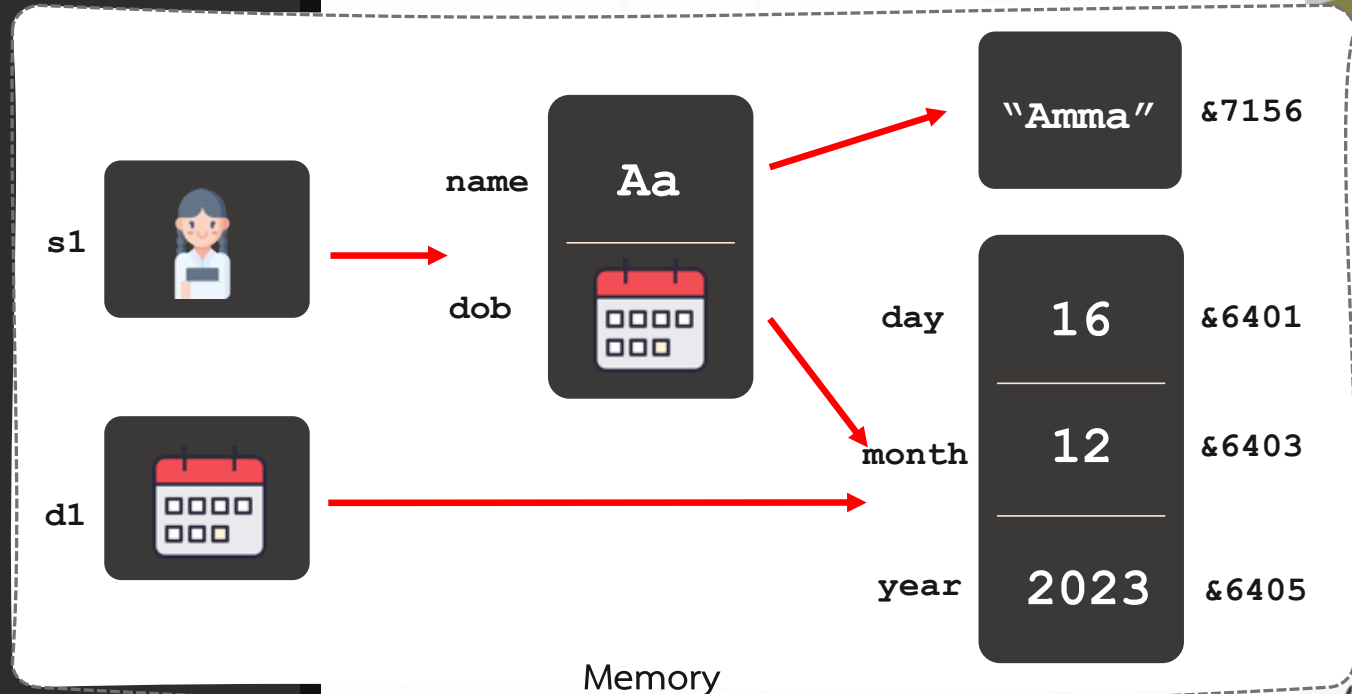
ตัวอย่างการส่งข้อมูลชนิดอ้างอิงเข้าเมธอดของอ็อบเจกต์

```
public class Main {  
    public void change(Dog b) {  
        b.name = "me";  
    }  
    public static void main(String[] args) {  
        Dog d = new Dog();  
        d.name = "John";  
        System.out.println(d.name);  
        new Main().change(d);  
        System.out.println(d.name);  
    }  
}  
  
public class Dog { # class Dog  
    public String name; # Attribute  
    public void printName() { # Method  
        System.out.println(name);  
    }  
}
```

Output:
John
me

การส่งข้อมูลชนิดอ้างอิงเข้าเมธอดของอ็อบเจกต์

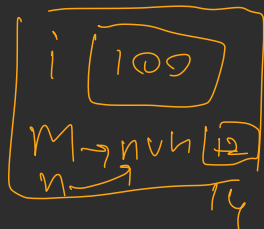
```
public class DOB {  
    public int day, month, year;  
    public void showInfo() {  
        System.out.println(day + "/" + month + "/" + year);  
    }  
}  
  
public class Student {  
    public String name; # String Attribute in class  
    public DOB dob; # Composite Attribute  
    public void setDOB(DOB d) {  
        dob = d;  
    }  
}  
  
public class Main {  
    public static void main(String args[]) {  
        Student s1 = new Student();  
        DOB d1 = new DOB();  
        s1.setDOB(d1);  
        s1.dob.day = 16;  
        s1.dob.month = 12;  
        s1.dob.year = 2023;  
        s1.name = "Amma";  
    }  
}
```



ตัวอย่าง

```
public class Main {  
    public int num;  
    public void addOne(int i){  
        i += 1;  
    }  
    public void addTwo(Main m){  
        m.num += 2;  
    }  
}
```

```
public static void main(String[] args) {  
    int i = 100;  
    Main m = new Main();  
  
    // Step 1  
    System.out.println(" i before : " + i);  
    m.addOne(1);  
    System.out.println(" i after : " + i);  
}
```



i before : 100
i after : 100

```
// Step 2  
m.num = 10;  
System.out.println(" obj m before : " + m.num);  
m.addTwo(m);  
System.out.println(" obj m after : " + m.num);
```

```
// Step 3  
Main n = m;  
System.out.println(" obj m before : " + m.num);  
System.out.println(" obj n before : " + n.num);  
m.addTwo(n);  
System.out.println(" obj m after : " + m.num);  
System.out.println(" obj n after : " + n.num);
```

```
}  
}
```

การส่งค่าเข้าไปในเมธอด (Argument)

เมธอดใด ๆ อาจมี argument สำหรับรับค่ามากกว่าหนึ่งตัว แต่การเรียกใช้เมธอดเหล่านี้จะต้องส่ง argument ที่มีชนิดข้อมูลที่สอดคล้องกันและมีจำนวนเท่ากัน

```
public class NumericalSample {  
    public void calMax(int i, double d) {  
        if (i > d) {  
            System.out.println("Max = "+i);  
        } else {  
            System.out.println("Max = "+d);  
        }  
    }  
  
    public static void main(String args[]) {  
        NumericalSample obj = new NumericalSample();  
        obj.calMax(3, 4.0);  
    }  
}
```

การส่งค่าเข้าไปในเมธอด (Argument)

ชนิดข้อมูลของ argument ที่จะส่งผ่านไปยังเมธอด ไม่จำเป็นที่จะต้องเป็นชนิดข้อมูลเดียวกัน แต่ต้องเป็นชนิดข้อมูลที่สามารถแปลงข้อมูลให้กว้างขึ้นได้โดยอัตโนมัติ

```
public class NumericalSample {  
    public void calMax(int i, double d) {  
        if (i > d) {  
            System.out.println("Max = "+i);  
        } else {  
            System.out.println("Max = "+d);  
        }  
    }  
    public static void main(String args[]) {  
        NumericalSample obj = new NumericalSample();  
        obj.calMax(3, 4);  
    }  
}  
Max = 4.0
```


การส่งค่าเข้าไปในเมธอด (Argument)

ชนิดข้อมูลของ argument ที่จะส่งผ่านไปยังเมธอด ไม่จำเป็นที่จะต้องเป็นชนิดข้อมูลเดียวกัน แต่ต้องเป็นชนิดข้อมูลที่สามารถแปลงข้อมูลให้กว้างขึ้นได้โดยอัตโนมัติ

```
public class NumericalSample {  
    public void calMax(int i, double d) {  
        if (i > d) {  
            System.out.println("Max = "+i);  
        } else {  
            System.out.println("Max = "+d);  
        }  
    }  
    public static void main(String args[]) {  
        NumericalSample obj = new NumericalSample();  
        obj.calMax(3.0, 4.0);  
    }  
}
```

```
Main.java:11: error: incompatible types: possible lossy conversion from double to int  
        obj.calMax(3.0,4.0);
```

การรับค่าที่ส่งกลับมาจากเมธอด

* 1 Method
return ได้ มา 1 ตัว 1

- เมธอดใดๆของคลาสสามารถที่จะมีค่าที่ส่งกลับมาได้ ซึ่งชนิดข้อมูลของค่าที่จะส่งกลับอาจเป็นชนิดข้อมูลแบบพื้นฐาน หรือเป็นชนิดข้อมูลแบบอ้างอิง

* หน้าทีของ return
1) Method จบลงทำงาน → กลับไปหาที่เรียกให้
2) สิ่งที่จะส่งกลับจะ ถูก replace Value เป็นค่าอะไร

- เมธอดที่มีค่าที่จะส่งกลับจะต้องมีคำสั่ง return ซึ่งจะระบุค่าที่ส่งกลับโดยมีรูปแบบดังนี้

```
return value;
```

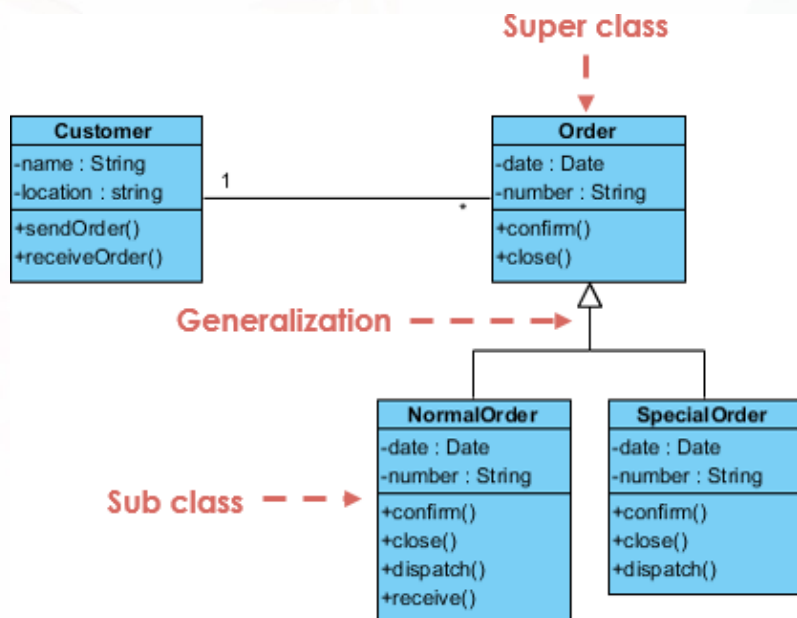
- คำสั่งที่เรียกใช้เมธอด อาจจะรับค่าที่ส่งกลับมาเก็บไว้ในตัวแปรหรือเป็นตัวถูกดำเนินการในนิพจน์ ตัวอย่างเช่น

```
Student s1 = new Student();  
s1.setGPA(3.2);  
double d = s1.getGPA();  
System.out.println("GPA:" + d);
```

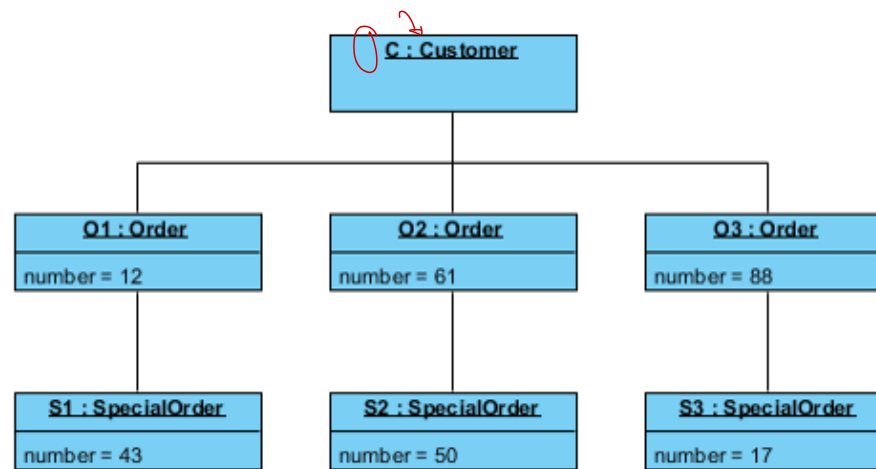
```
public class Student {  
    private double gpa;  
    public void setGPA(double GPA) {  
        gpa = GPA;  
    }  
    public double getGPA() {  
        return gpa;  
    }  
}
```

Unified Modeling Language (UML)

Unified Modeling Language (UML) เป็นภาษาที่สามารถนำรูปภาพฟิคมาจำลองโปรแกรมเชิงอ็อบเจกต์ได้ ซึ่งประกอบด้วย 2 ส่วนคือ (1) ไดอะแกรมของคลาส (Class Diagram) และ (2) ไดอะแกรมของอ็อบเจกต์ (Object Diagram)



(Class Diagram)

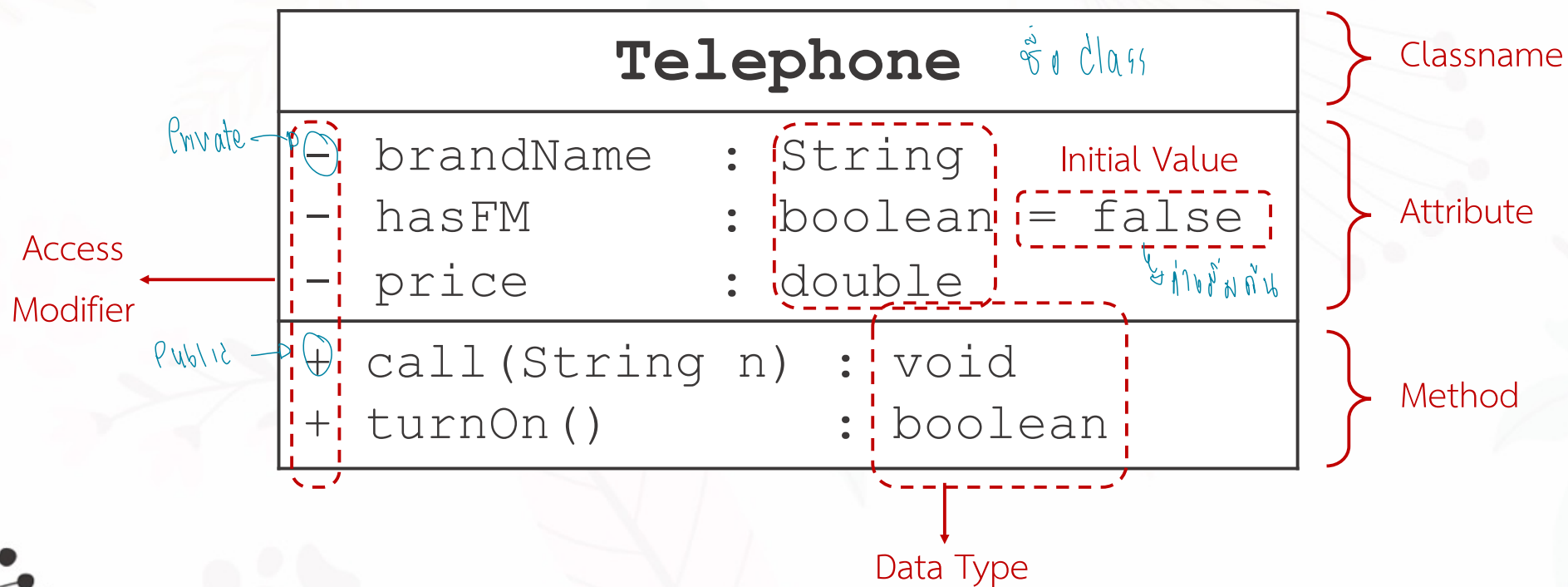


(Object Diagram)

อ้างอิง <https://guides.visual-paradigm.com/unveiling-uml-navigating-the-differences-between-object-diagrams-and-class-diagrams/>

ไต่อะแกรมของคลาส

ไต่อะแกรมของคลาส คือ เป็นสัญลักษณ์ที่ใช้แสดงรายละเอียดของคลาส ซึ่งประกอบด้วย 3 ส่วนหลัก ได้แก่ (1) ชื่อของคลาส (2) คุณลักษณะภายในคลาส และ (3) เมธอดภายในคลาส



การเขียนโปรแกรมจากไดอะแกรมของคลาส

Class Diagram

Telephone	
- brandName	: String
- hasFM	: boolean
- price	: double
+ call(String n)	: void
+ turnOn()	: boolean



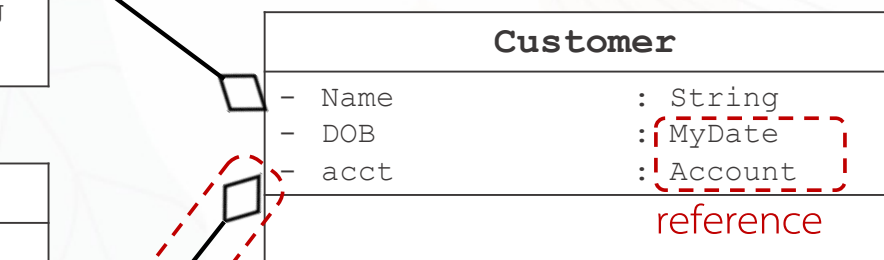
```
public class Telephone {  
    private String brandName;  
    private boolean hasFM;  
    private double price;  
  
    public void call(String n) {  
        ...  
    }  
  
    public boolean turnOn() {  
        ...  
    }  
}
```

ไดอะแกรมของคลาส

* 99 211 1 78
0 0

Account	
# balance	: double
# name	: String
+ deposit(double balance)	: void
+ withdraw(double balance)	: void
+ setName(String name)	: void
+ getName()	: String
+ showInfo()	: void

MyDate	
- day	: int
- month	: int
- year	: int
+ setDay(int day)	: void
+ getDay()	: int
+ setMonth(int month)	: void
+ getMonth()	: int
+ setYear(int year)	: void
+ getYear()	: int
+ showDate()	: void



} Classname
} Attribute

Composition
Relation

ชนิดข้อมูลนามธรรม

ชนิดข้อมูลแบบนามธรรม (Abstract Data Type)

คือ ชนิด หรือ คลาสของวัตถุที่ถูกนิยามด้วยเซตของข้อมูลและตัวดำเนินการ ซึ่งเป็นการนิยามหลักการทำงานและพฤติกรรมของข้อมูลเท่านั้น แต่ไม่ระบุถึงวิธีการสร้างหรือโปรแกรมเพื่อใช้งานจริง ดังนั้นชนิดข้อมูลดังกล่าวจึงถูกเรียกว่า นามธรรม

Type

5 3 1
7 4 9 0 2

Operations

× ÷ + -

OOP vs Data Structure

Abstract Data Type

Type

Operations

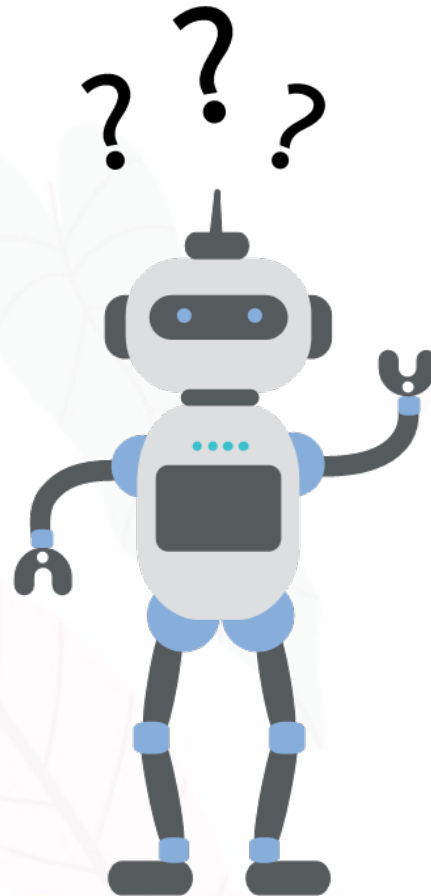
Data Structure

Class

Attribute

Method

Object Oriented Programming





THANKS !